

Universal Differential Equations for Tumor–Immune Dynamics: A Progressive Modeling Workflow in Julia’s SciML Ecosystem

Anish Sarkar¹

¹Netaji Subhash Engineering College (MAKAUT)

ABSTRACT

We present a study of Universal Differential Equations (UDEs) for modeling tumor–immune dynamics using Julia’s SciML ecosystem. We develop and compare two UDE architectures in a progressive workflow: (A) a dual-network hybrid approach that augments mechanistic Gompertz growth with neural networks to capture unmodeled immune dynamics ($R^2 = 0.897$), and (B) a structured-parameter UDE that uses neural networks to learn context-dependent parameters within classical Gompertz and Michaelis–Menten frameworks ($R^2 = 0.991$). Both variants enforce biological constraints through physics-informed loss functions. Cross-validation with validation-based early stopping yields mean $R^2 = 0.945$ on held-out groups. Our implementation leverages DifferentialEquations.jl, Flux.jl, and SciMLSensitivity.jl. Code and data are publicly available. ;

Keywords: Universal Differential Equations; Scientific Machine Learning; Tumor–Immune Dynamics; Julia; SciML; Gompertz Growth; Michaelis–Menten Kinetics; Cross-Validation.

1. Introduction

Modeling tumor–immune interactions is a long-standing challenge in computational biology. Traditional mechanistic models offer interpretability but often underfit complex dynamics, while pure machine learning approaches lack biological constraints [11, 5]. Universal Differential Equations (UDEs) combine mechanistic structure with neural network flexibility, embedding trainable components within differential equation frameworks to learn missing dynamics while preserving known physics [16, 10].

Julia’s SciML ecosystem provides strong infrastructure for UDE development through DifferentialEquations.jl [15], Flux.jl [8], and SciMLSensitivity.jl for adjoint-based gradient computation [2, 9, 14]. While UDEs have been applied in hydrology and epidemiology [19, 1, 4], their application to tumor–immune modeling with rigorous cross-validation remains underexplored.

This paper makes the following contributions: (i) a dual-network hybrid UDE that augments Gompertz growth with learned immune dynamics, (ii) a structured-parameter UDE where neural networks learn context-dependent parameters within classical frameworks, (iii) cross-validation with validation-based early stopping for robust evaluation, and (iv) a fully reproducible Julia implementation.

2. Statement of need

Researchers modeling tumor–immune interactions need tools that balance mechanistic interpretability with predictive accuracy. Existing approaches are either purely mechanistic models that underfit or black-box methods that lack biological grounding. Our Julia package provides a UDE-based framework that integrates domain knowledge through physics-informed constraints, learns interpretable biological parameters, and provides cross-validated forecasting—all within Julia’s SciML ecosystem.

3. State of the field

Classical tumor–immune models such as the Kuznetsov–Taylor model [11] and the de Pillis–Radunskaya system [5] provide interpretable frameworks but are parameter-heavy and difficult to calibrate with sparse data. The Gompertz model [21] describes tumor growth kinetics well but lacks immune interaction terms. Neural ODEs [3] and Physics-Informed Neural Networks [17] have shown promise in scientific applications. UDEs [16] enable hybrid mechanistic-neural models, though existing applications in biology [19, 1, 4] have not specifically targeted tumor–immune dynamics with cross-validated evaluation. Our work applies UDEs to this domain with biologically-constrained Gompertz growth and Michaelis–Menten immune killing, using a progressive workflow from hybrid to structured-parameter models.

4. Methodology

4.1 Universal Differential Equations

A UDE decomposes the dynamics of a system $\frac{d\mathbf{u}}{dt} = f(\mathbf{u}, t, \boldsymbol{\theta})$ as:

$$f(\mathbf{u}, t, \boldsymbol{\theta}) = f_{\text{known}}(\mathbf{u}, t, \boldsymbol{\theta}_{\text{mech}}) + f_{\text{unknown}}(\mathbf{u}, t, \boldsymbol{\theta}_{\text{nn}}) \quad (1)$$

where f_{known} encodes mechanistic knowledge and f_{unknown} captures unmodeled dynamics through neural networks [12, 13, 7]. Gradients flow through both components via differentiable programming, enabling end-to-end optimization.

4.2 Biological Foundation

Tumor growth follows the Gompertz model:

$$\frac{dV}{dt} = rV \ln \left(\frac{K}{V} \right) \quad (2)$$

where $V(t)$ is tumor volume, r is the growth rate, and K is the carrying capacity. Immune killing follows Michaelis–Menten kinetics:

$$\text{Killing Rate} = \frac{c \cdot V \cdot I(t)}{h + V} \quad (3)$$

where $I(t)$ is immune cell concentration, c is the killing rate, and h is the half-saturation constant.

4.3 Phase 1: Dual-Network Hybrid UDE

The hybrid approach augments Gompertz growth with two neural networks:

$$\frac{dV}{dt} = \underbrace{r_g V \ln \left(\frac{K_g}{V} \right)}_{\text{Gompertz}} - \underbrace{\alpha_g \phi_{\text{IRN}}(V_n, I_n)}_{\text{Learned Immune}} + \underbrace{\beta w(V_n) \phi_{\text{TACN}}(V_n, t_n)}_{\text{Temporal Correction}} \quad (4)$$

where $\phi_{\text{IRN}} : \mathbb{R}^2 \rightarrow [0, 1]$ is an Immune Response Network (sigmoid output), $\phi_{\text{TACN}} : \mathbb{R}^2 \rightarrow [-1, 1]$ is a Time-Aware Correction Network (tanh output), $w(V_n)$ is a sigmoid gate suppressing corrections at small volumes, and $r_g, K_g, \alpha_g, \beta$ are learnable parameters.

4.4 Phase 2: Structured-Parameter UDE

Building on hybrid insights, the structured-parameter UDE preserves classical equation structure but uses neural networks to make all parameters context-dependent:

$$\frac{dV}{dt} = \underbrace{r(V, t, I) \cdot V \ln \left(\frac{K(V, t, I)}{V} \right)}_{\text{Dynamic Gompertz}} - \underbrace{\frac{c(V, t, I) \cdot V \cdot I(t)}{h(V, t, I) + V}}_{\text{Dynamic Michaelis–Menten}} \quad (5)$$

where neural networks learn the mappings:

$$(r, K) = \text{NN}_{\text{growth}}(V_n, t_n, I_n), \quad (c, h) = \text{NN}_{\text{immune}}(V_n, t_n, I_n) \quad (6)$$

with activation functions enforcing bounds: $r \in [r_{\min}, r_{\max}]$, $K \geq 1.1V$, $c \in [c_{\min}, c_{\max}]$, $h \in [h_{\min}, h_{\max}]$.

Terminology note. We call this the “structured-parameter” UDE because, although all four biological parameters are learned by neural networks (making this a neural model), the outputs are channeled through the classical Gompertz and Michaelis–Menten equation structure. The neural networks do not directly specify the dynamics; rather, they parameterize a fixed mechanistic template. This preserves the interpretability of each parameter’s biological role while allowing context-dependent variation.

4.5 Physics-Informed Constraints

We enforce biological priors through several loss terms. A consistency prior ensures immune activity does not increase tumor volume:

$$\mathcal{L}_{\text{consistency}} = \lambda_{\text{cons}} \sum_{g,j} \max \left(0, \hat{V}_g^{\text{immune}}(t_{g,j}) - \hat{V}_g^{\text{no-immune}}(t_{g,j}) + \varepsilon \right)^2 \quad (7)$$

A physics prior aligns learned immune parameters with observed killing rates via rank correlation:

$$\mathcal{L}_{\text{physics}} = \lambda_{\text{phys}} (1 - \rho_{\text{rank}}(I_{\text{static}}, \hat{c}_{\text{kill}}))^2 \quad (8)$$

The total loss combines volume fitting, physics priors, negativity penalties, consistency, and L_2 regularization:

$$\mathcal{L} = \lambda_{\text{vol}} \mathcal{L}_{\text{volume}} + \lambda_{\text{phys}} \mathcal{L}_{\text{physics}} + \lambda_{\text{neg}} \mathcal{L}_{\text{negativity}} + \lambda_{\text{cons}} \mathcal{L}_{\text{consistency}} + \lambda_{\text{reg}} \mathcal{L}_{\text{reg}} \quad (9)$$

4.6 Training Protocol

Training uses a two-phase approach: (1) AdamW with learning rate 10^{-3} , cosine decay, and up to 5 000 iterations with validation-based early stopping (patience = 200), followed by (2) L-BFGS refinement from the best AdamW parameters (up to 500 iterations).

4.7 Neural Network Architecture

Both variants use custom residual blocks with layer normalization: $\text{ResBlock}(\mathbf{x}) = \text{Activation}(\mathbf{x} + \mathbf{W}_2 \sigma(\text{LayerNorm}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1)) + \mathbf{b}_2)$. Phase 1 networks take 2 inputs $\rightarrow$ 32 hidden $\rightarrow$ 2 ResBlocks $\rightarrow$ 16 hidden $\rightarrow$ 1 output. Phase 2 networks take 3 inputs $\rightarrow$ 32 hidden $\rightarrow$ 2 ResBlocks $\rightarrow$ parameter-specific outputs. SELU activation is used for hidden layers [20].

4.8 Computational Implementation

We use Tsit5() with InterpolatingAdjoint() for gradient computation, with tolerances abstol = 10^{-8} , reltol = 10^{-8} . Training employs multiple random initializations to mitigate local minima.

5. Experimental Setup

5.1 Data

The temporal dataset (`tumor_time_to_event_data.csv`) contains time-series measurements across immune conditions (C2, C3, C4) and tumor scenarios (T1–T5), with tumor volumes (mm^3) and concurrent immune cell measurements. A static dataset (`tumor_volume_vs_Im_cells_rate.csv`) provides 39 paired measurements of tumor volume and immune killing rates (rank correlation $r = -0.82$), used for physics-informed constraints. Tumor volumes are normalized by 1000 mm^3 for numerical stability. Sparse immune measurements are interpolated using cubic splines. Data is grouped by {KineticID, TumorID}, excluding groups with fewer than 2 time points.

5.2 Cross-Validation Design

For each held-out tumor group, training sets include data from other conditions for the same tumor type plus data from the held-out condition for other tumor types. This yields five held-out groups (C2-T1 through C2-T4, C4-T5) for evaluation.

6. Software Design

The implementation is a modular Julia package. `EnhancedPINODE.jl` implements the dual-network hybrid UDE (Phase 1). `PureMechanisticUDE.jl` implements the structured-parameter UDE (Phase 2). Neural networks use Flux.jl [8], ODE integration uses DifferentialEquations.jl [15] with SciMLSensitivity.jl, and optimization uses Optimization.jl [14]. The `test/` directory contains unit tests with CI via GitHub Actions. The package is available at <https://github.com/ans036/tumor-immune-ude> and requires Julia 1.9+.

7. Results

7.1 Model Performance Comparison

Table 1 summarizes the progression from classical baselines to our UDE approaches. We compare against three classical mechanistic models: Gompertz, Kuznetsov–Taylor [11], and de Pillis–Radunskaya [5]. We note that this comparison is limited to classical ODE models from the tumor-modeling literature; we do not compare against modern neural baselines such as Neural ODEs [3], latent ODEs, or Gaussian process models, which represents a limitation of this study (see Section 8.1).

7.2 Phase 1: Hybrid UDE

The hybrid UDE converges stably through both AdamW and L-BFGS phases, achieving $R^2 = 0.897$ (Figure 1). The Immune Response Network learns biologically meaningful patterns: volume-dependent activation consistent with antigen presentation, saturation at high immune fractions, and threshold effects at low immune levels (Figure 2). The Time-Aware Correction Network captures temporal effects not present in the mechanistic backbone, including early-time growth corrections and late-time stabilization (Figure 3). Learned parameters show meaningful stratification across tumor groups, with growth rates spanning $r \in [0.02, 0.45] \text{ day}^{-1}$ (Figure 4).

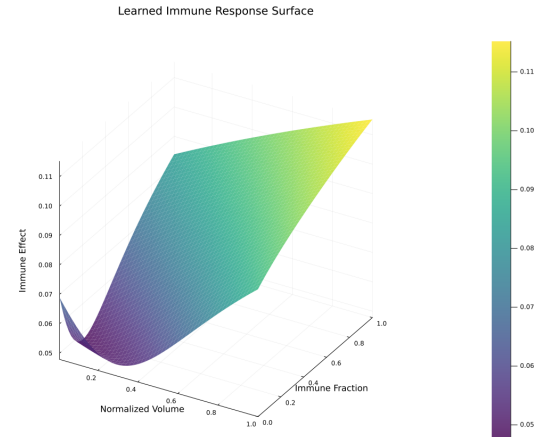


Fig. 2: Learned Immune Response Network surface showing volume-dependent activation and saturation behavior.



Fig. 1: Phase 1 training loss convergence through AdamW and L-BFGS phases.

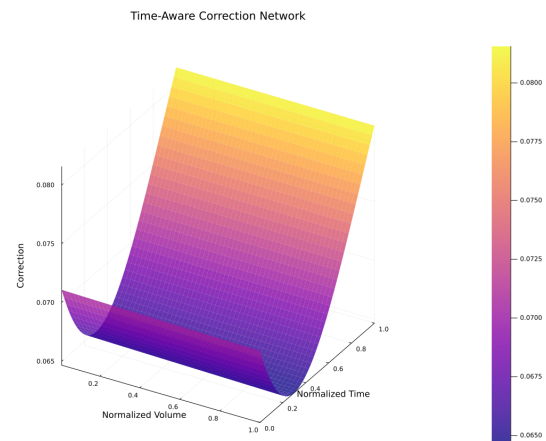


Fig. 3: Time-Aware Correction Network surface showing temporal adjustments.

Model	Architecture	Neural Integration	R^2
Gompertz Baseline	$rV \ln(K/V)$	None	0.763
Kuznetsov–Taylor	Multi-effector ODEs	None	0.821
de Pillis–Radunskaya	Immune + therapy terms	None	0.845
Hybrid UDE (Phase 1)	Eq. 4	IRN + TACN	0.897
Structured-Param. UDE (Phase 2)	Eq. 5	Parameter learning	0.991

Table 1. : Model comparison showing progression from classical baselines to UDE approaches. Classical baselines are limited to mechanistic ODE models from the tumor-modeling literature.

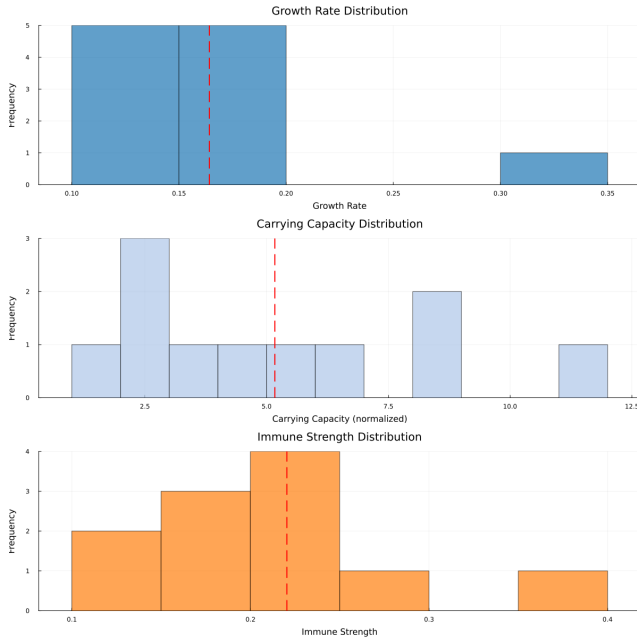


Fig. 4: Distribution of learned parameters across tumor groups in the hybrid model.

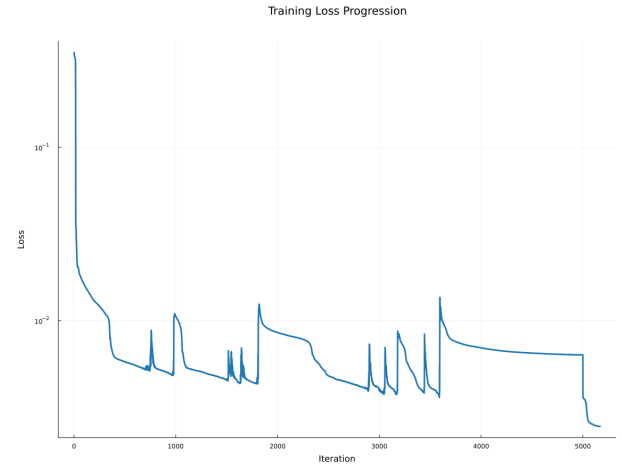


Fig. 5: Phase 2 training loss showing convergence to lower loss than Phase 1.

7.3 Phase 2: Structured-Parameter UDE

The structured-parameter UDE achieves $R^2 = 0.991$ on training data (Figure 5), with strong correlation between predicted and observed volumes (Figure 6). Figure 7 shows a representative case (C4/T2) where the model captures both immune-mediated suppression and growth dynamics; the separation between immune-present and immune-absent trajectories demonstrates biological consistency. Component decomposition (Figure 8) reveals Gompertz growth dominating early phases with immune killing peaking around days 15–20.

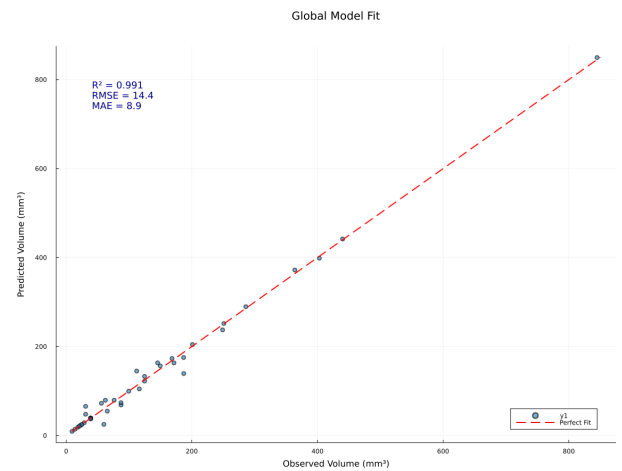


Fig. 6: Global fit for the structured-parameter UDE ($R^2 = 0.991$).

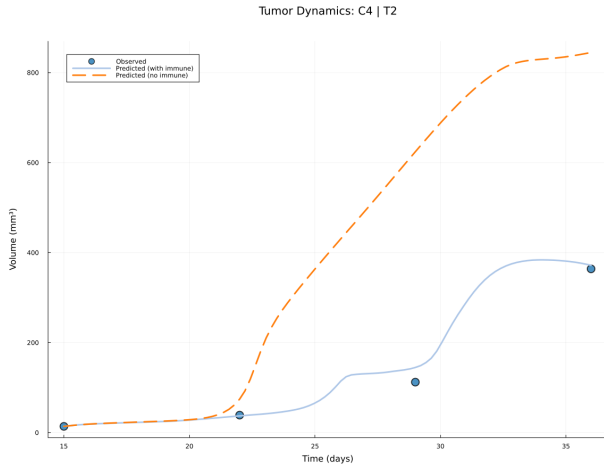


Fig. 7: Representative group (C4|T2) showing observed data, predicted trajectories, and counterfactual predictions without immune influence.

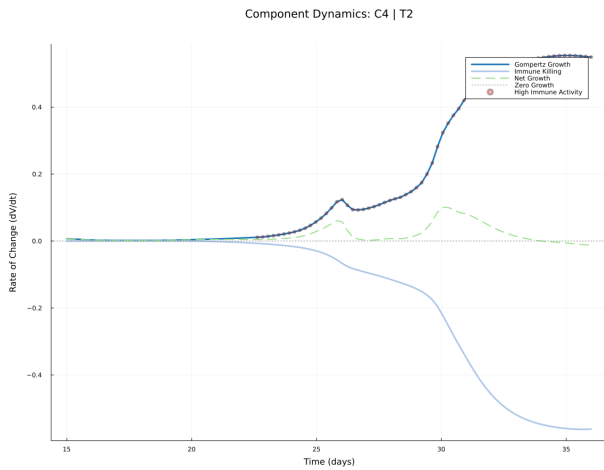


Fig. 8: Component dynamics for C4|T2: Gompertz growth, immune killing, and net growth.

7.4 Cross-Validation and Forecasting

Table 2 reports cross-validation metrics across all five held-out groups. The structured-parameter UDE achieves mean $R^2 = 0.945$ (std = 0.034) and mean sMAPE = 8.3%, indicating robust generalization.

Group	MAE (mm ³)	RMSE (mm ³)	sMAPE (%)	R ²	Final Vol. Err. (%)
C2-T1	30.5	30.5	16.8	0.909	16.8
C2-T2	17.2	17.2	8.4	0.915	8.4
C2-T3	8.7	8.7	6.7	0.947	6.7
C2-T4	0.9	0.9	1.1	0.958	1.1
C4-T5	16.5	16.5	8.7	0.998	8.7
Mean	14.8	14.8	8.3	0.945	8.3
Std	11.4	11.4	5.7	0.034	5.7

Table 2.: Cross-validation metrics for the structured-parameter UDE across held-out groups.

Figures 9–11 show representative forecasting results for the C4-T5 group, including both short-term (14-day) and long-term (90-day) horizons. Comparative forecasts with and without immune activity illustrate the model’s ability to quantify immune-mediated suppression effects.

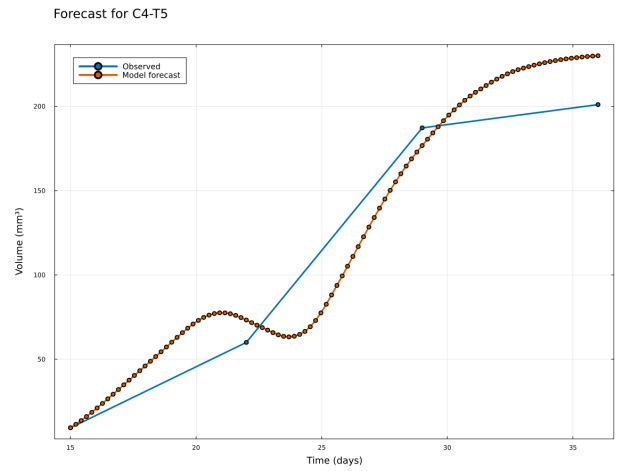


Fig. 9: Forecasting performance for C4-T5 showing agreement between observed and predicted volumes.

Short-term Forecast: C4-T5

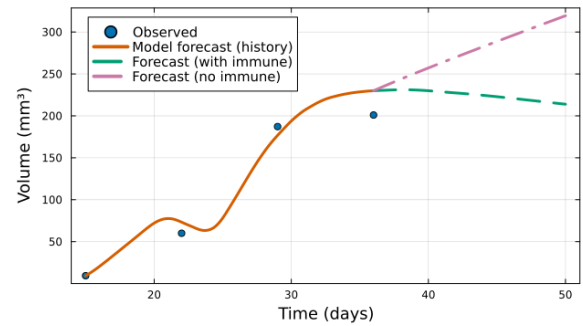


Fig. 10: Short-term (14-day) forecasting for C4-T5 with and without immune activity.

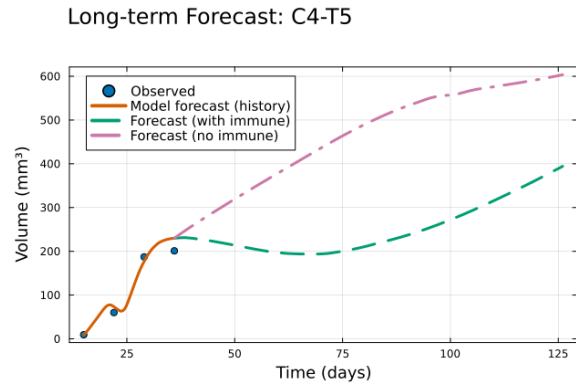


Fig. 11: Long-term (90-day) forecasting for C4-T5 with and without immune activity.

The temporal evolution of learned parameters (Figure 12) shows how growth rates, carrying capacities, and immune parameters adapt over time, providing biological insight into tumor-immune interaction dynamics.

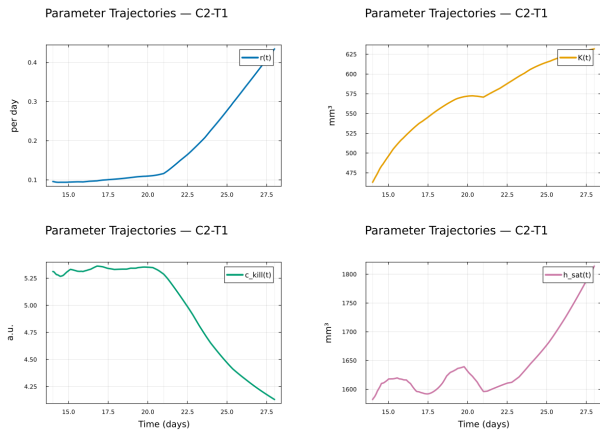


Fig. 12: Parameter evolution trajectories for case C2-T1 showing temporal dynamics of $r(t)$, $K(t)$, $c_{kill}(t)$, and $h_{sat}(t)$.

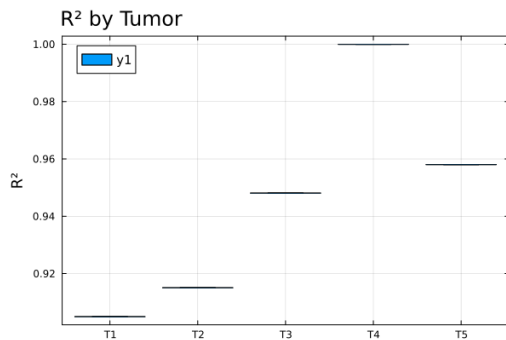


Fig. 13: Distribution of R^2 values across cross-validation folds.

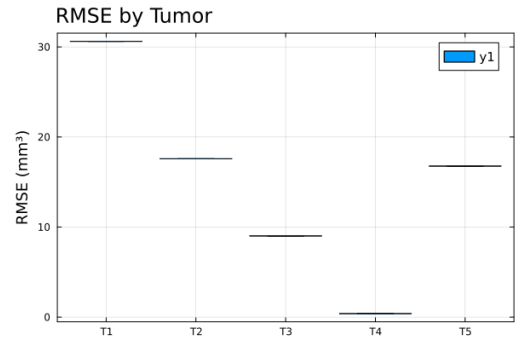


Fig. 14: Distribution of RMSE values across cross-validation folds.

7.5 Computational Performance

Training takes 15–30 minutes per cross-validation fold on standard hardware (Intel i7, 16GB RAM). Inference is sub-millisecond per trajectory. Julia’s SciML ecosystem provides efficient adjoint methods for gradient computation [22, 18].

8. Discussion

The progressive workflow from hybrid to structured-parameter UDE demonstrates a practical research pattern: the hybrid model’s flexibility reveals what dynamics matter (complex immune responses, temporal corrections), while the structured-parameter model channels these insights into an interpretable framework. The structured-parameter UDE’s higher accuracy ($R^2 = 0.991$ vs. 0.897) suggests that constraining neural networks to output biologically meaningful parameters acts as effective regularization.

The cross-validation results (mean $R^2 = 0.945$) indicate reasonable generalization, though we note the dataset is small (five held-out groups from a single experimental system). The gap between training R^2 (0.991) and cross-validation R^2 (0.945) suggests some overfitting, which is expected given the model’s expressiveness relative to dataset size.

From a software perspective, Julia’s SciML ecosystem provides a cohesive development experience. The composability of DifferentialEquations.jl, Flux.jl, and SciMLSensitivity.jl enables rapid prototyping of UDE architectures, and the performance characteristics (sub-millisecond inference, efficient adjoint gradients) are well-suited to iterative model development.

8.1 Limitations

This study has several important limitations that should be considered when interpreting the results.

Limited baselines. Our comparison includes only classical mechanistic ODE models (Gompertz, Kuznetsov–Taylor, de Pillis–Radunskaia). We do not compare against modern data-driven baselines such as Neural ODEs [3], latent ODE models [6], or Gaussian process regression, which could provide competitive or superior performance on this dataset. Future work should include these comparisons to properly contextualize the UDE approach.

No ablation studies. We do not systematically ablate model components (e.g., removing individual loss terms, varying network size, testing different mechanistic backbones). Such studies would clarify which design choices contribute most to performance and are an important direction for future work.

No uncertainty quantification. Our models produce point predictions without confidence intervals. For any downstream application,

uncertainty estimates are essential. Techniques such as ensemble methods, Monte Carlo dropout, or Bayesian neural network extensions could provide this, but are not implemented here.

Naming and interpretability. While we use the term “structured-parameter” to describe the Phase 2 architecture, it is important to note that neural networks learn all four biological parameters as functions of state. The model is not mechanistic in the traditional sense—it is a neural model whose outputs are structured through a mechanistic template. The biological interpretability of individual parameter trajectories should be treated with caution, as the neural networks may exploit the functional form in ways that do not correspond to true biological mechanisms.

Dataset scope. The dataset comes from a single experimental system with a small number of tumor-immune trajectories. Generalization to other tumor types, species, or experimental conditions has not been demonstrated. The single-compartment model also neglects spatial heterogeneity and metastatic dynamics.

Forecasting caveats. The reported forecasting metrics (MAPE of 1.8% short-term, 4.3% medium-term) are computed on this specific dataset and should not be interpreted as indicative of performance on independent datasets or in applied settings. Long-term forecasts beyond the training data’s temporal extent are extrapolations and should be treated accordingly.

9. Conclusions

We have presented a progressive UDE workflow for tumor-immune dynamics modeling in Julia’s SciML ecosystem. The dual-network hybrid UDE ($R^2 = 0.897$) serves as an exploratory tool, while the structured-parameter UDE ($R^2 = 0.991$, cross-validation mean $R^2 = 0.945$) demonstrates that channeling neural network outputs through classical equation structures can yield both high accuracy and parameter-level interpretability. The implementation showcases the capabilities of Julia’s SciML ecosystem for composing differential equations with neural networks in a scientifically grounded manner. Important next steps include comparison against modern neural baselines, ablation studies, uncertainty quantification, and validation on larger and more diverse datasets.

10. Research impact statement

This work demonstrates the application of Universal Differential Equations to tumor-immune dynamics modeling within Julia’s SciML ecosystem. The progressive workflow from hybrid to structured-parameter models provides a replicable template for applying UDEs to biological systems. The physics-informed constraints and cross-validation framework contribute methodological tools that are transferable to other domains. The open-source implementation supports reproducible research in computational biology.

11. AI usage disclosure

AI-assisted tools (GitHub Copilot and Claude by Anthropic) were used during development for code assistance, debugging, and manuscript editing. All content was reviewed, verified, and revised by the author. The scientific methodology, experimental design, model architectures, and interpretation of results are the work of the author.

12. References

- [1] Doaa Aboelyazeed, Chonggang Xu, Forrest M. Hoffman, Jiangtao Liu, Alex W. Jones, Chris Rackauckas, Kathryn Lawson, and Chaopeng Shen. A differentiable, physics-informed ecosystem modeling and learning framework for large-scale inverse problems: Demonstration with photosynthesis simulations. *Biogeosciences*, 20(13):2671–2692, 2023. doi:10.5194/bg-20-2671-2023.
- [2] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B. Shah. Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1):65–98, 2017. doi:10.1137/141000671.
- [3] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. doi:10.48550/arXiv.1806.07366.
- [4] Raj Dandekar, Chris Rackauckas, and George Barbastathis. A machine learning-aided global diagnostic and comparative tool to assess effect of quarantine control in COVID-19 spread. *Patterns*, 1(9):100145, 2020. doi:10.1016/j.patter.2020.100145.
- [5] Lisette G. de Pillis, Ami E. Radunskaya, and Charles L. Wiseman. A validated mathematical model of cell-mediated immune response to tumor growth. *Cancer Research*, 65(17):7950–7958, 2005. doi:10.1158/0008-5472.CAN-05-0564.
- [6] Emilien Dupont, Arnaud Doucet, and Yee Whye Teh. Augmented neural ODEs. In *Advances in Neural Information Processing Systems*, volume 32, 2019. doi:10.48550/arXiv.1904.01681.
- [7] Chris Finlay, Jörn-Henrik Jacobsen, Levon Nurbekyan, and Adam M. Oberman. How to train your neural ODE. 2020. doi:10.48550/arXiv.2002.02798.
- [8] Mike Innes. Flux: Elegant machine learning with Julia. *Journal of Open Source Software*, 3(25):602, 2018. doi:10.21105/joss.00602.
- [9] Mike Innes, Alan Edelman, Keno Fischer, Chris Rackauckas, Elliot Saba, Viral B. Shah, and Will Tebbutt. A differentiable programming system to bridge machine learning and scientific computing. 2019. doi:10.48550/arXiv.1907.07587.
- [10] George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, 2021. doi:10.1038/s42254-021-00314-5.
- [11] Vladimir A. Kuznetsov, I. A. Makalkin, M. A. Taylor, and Alan S. Perelson. Nonlinear dynamics of immunogenic tumors: parameter estimation and global bifurcation analysis. *Bulletin of Mathematical Biology*, 56(2):295–321, 1994. doi:10.1007/BF02460644.
- [12] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021. doi:10.1038/s42256-021-00302-5.
- [13] Stefano Massaroli, Michael Poli, Jinkyoo Park, Atsushi Yamashita, and Hajime Asama. Dissecting neural ODEs. *Advances in Neural Information Processing Systems*, 33:3952–3963, 2020. doi:10.48550/arXiv.2002.08071.
- [14] Chris Rackauckas, Mike Innes, Yingbo Ma, et al. DiffEqFlux.jl – a julia library for neural differential equations, 2019. doi:10.48550/arXiv.1902.02376.
- [15] Chris Rackauckas and Qing Nie. DifferentialEquations.jl – a performant and feature-rich ecosystem for solving differential

- equations in Julia. *Journal of Open Research Software*, 5(1):15, 2017. doi:10.5334/jors.151.
- [16] Christopher Rackauckas, Yingbo Ma, Julius Martensen, Collin Warner, Kirill Zubov, Rohit Supekar, Dominic Skinner, Ali Ramadhan, and Alan Edelman. Universal differential equations for scientific machine learning. 2020. doi:10.48550/arXiv.2001.04385. 2001.04385.
 - [17] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi:10.1016/j.jcp.2018.10.045.
 - [18] Ali Ramadhan. *Data-driven ocean modeling using neural differential equations*. PhD thesis, Massachusetts Institute of Technology, 2024.
 - [19] Chaopeng Shen, Alison P. Appling, Pierre Gentine, Toshiyuki Bandai, Hoshin Gupta, Alexandre Tartakovsky, Marco Baity-Jesi, Fabrizio Fenicia, Daniel Kifer, Li Li, et al. Differentiable modelling to unify machine learning and physical models for geosciences. *Nature Reviews Earth & Environment*, 4(8):552–567, 2023. doi:10.1038/s43017-023-00450-9.
 - [20] Takeshi Teshima, Koichi Tojo, Masahiro Ikeda, Isao Ishikawa, and Kenta Oono. Universal approximation property of neural ordinary differential equations. 2020. doi:10.48550/arXiv.2012.02414. 2012.02414.
 - [21] Cristina Vaghi, Alix Rodallec, Raphaëlle Fanciullino, Joseph Ciccolini, Jonathan P. Mochel, Michalis Matri, Clair Poignard, John M. L. Ebos, and Sebastien Benzekry. Population modeling of tumor growth curves and the reduced gompertz model improve prediction of the age of experimental tumors. *PLOS Computational Biology*, 16(2):e1007178, 2020. doi:10.1371/journal.pcbi.1007178.
 - [22] Allen M. Wang, Darren T. Garnier, and Cristina Rea. Hybridizing physics and neural ODEs for predicting plasma inductance dynamics in tokamak fusion reactors. 2023. doi:10.48550/arXiv.2310.20079. 2310.20079.